

CS3423 Compilers 2

Mini Assignment 3 – LLVM IR Exploration Report

Names: Krishnan R, Harikrishna

Roll Numbers: CS24BTECH11036, CS24BTECH11030

Platform: WSL2 (Ubuntu)

Clang Version: Clang version 17.0.6

LLVM Version: 17.0.6 (built from source: llvmorg-17.0.6 with clang and clang-tools-extra, X86 target)

Contents

1 LLVM Directory Layout	1
2 LLVM IR Study of Core C/C++ Constructs	1
2.1 C Code in Q1a.c	1
2.2 C Code in Q1b.c	2
2.3 C Code in Q1c.c	4
2.4 C Code in Q1d.c	5
3 Lowering of Data Structures to LLVM IR	6
3.1 C Code in Q2a.c	6
3.2 C Code in Q2bi.c (struct) and Q2bii.cpp (class)	7
3.3 C++ Code in Q2c_vector.cpp (std::vector)	9
3.4 C Code in Q2d_union.c (union)	9
3.5 C++ Code in Q2e_struct_class.cpp (struct vs class)	10
4 Floating-Point Representation in LLVM IR	11

Introduction

This report delves into the LLVM compilation pipeline using Clang 17. It involves generating an IR (Intermediate Representation) for various constructs, analysing control flow and data structures, and studying optimizations (mainly -O0 and -O1).

1 LLVM Directory Layout

The LLVM source (release/17.x of llvm-project) is divided into the following main directories:

llvm/: contains the core of LLVM. include/llvm holds the header files, lib/ holds their implementations (IR, Analysis, Transforms (optimization passes), CodeGen, Target), tools/ holds programs like llc, opt and lli, and test/ holds the regression tests.

clang/: contains the C-family frontend. lib/Lex, lib/Parse, lib/Sema and lib/AST handle lexing, parsing, semantic analysis and the AST, while lib/CodeGen converts the AST to LLVM IR (the .ll

files in this report).

clang-tools-extra/: contains extra tools built on Clang such as clang-tidy and clangd.

Other directories (cmake/, runtimes/, ...) hold build files and runtime libraries. Only clang and clang-tools-extra are enabled using `-DLLVM_ENABLE_PROJECTS="clang-tools-extra;clang"`.

Flow followed in this report: C/C++ source → clang frontend → LLVM IR (.ll) → llc → assembly.

2 LLVM IR Study of Core C/C++ Constructs

2.1 C Code in Q1a.c

```
#include <stdio.h>
int a = 5; // (i) Global Integer
int main(){
    int b = 10; // (ii) A Local Integer
    int c = 3;  // (ii) A Local Integer
    int d = a + b * c; // (iii) A local Integer initialized to a + b * c
}
```

IR Code Snippets in Q1a.ll:

```
@a = dso_local global i32 5, align 4 ; (i) Global Integer a = 5
%1 = alloca i32, align 4             ; (ii) allocate space for Local Integer b
%2 = alloca i32, align 4             ; (ii) allocate space for Local Integer c
%3 = alloca i32, align 4
store i32 10, ptr %1, align 4        ; (ii) Initialize a Local Integer b = 10
store i32 3, ptr %2, align 4         ; (ii) Initialize a Local Integer c = 3
%4 = load i32, ptr @a, align 4       ; load a
%5 = load i32, ptr %1, align 4       ; load b
%6 = load i32, ptr %2, align 4       ; load c
%7 = mul nsw i32 %5, %6              ; b * c
%8 = add nsw i32 %4, %7              ; a + b * c
store i32 %8, ptr %3, align 4        ; (iii) Store a + b * c in d
```

Explanation of why local variables appear as an alloca instruction rather than as named registers:

In LLVM IR, named registers follow the SSA (Static Single Assignment) form, meaning each register is assigned exactly once and does not have a memory address. On the other hand, C variables are real variables stored in memory (stack) which can be done using alloca (which allocates memory on stack to the variable), have an address, can be modified multiple times throughout the program. Therefore, Clang represents local variables using alloca instructions, which allocate space on the stack, and uses load and store instructions to read and modify their values. This allows LLVM IR to correctly model the mutable nature of variables in C.

2.2 C Code in Q1b.c

```
#include <stdio.h>
int main(){
    int a = 2, b = 3;
    // (i) If-Else Condition
    if (a == 2){
        a = 2;
    }
    else{
        a = 2;
    }
    // (ii) Switch Case Condition
```

```

switch(a){
    case 1: a = 2; break;
    case 2: a = 2; break;
    default: a = 2; break;
}
// (iii) For loop
for (int i = a; i < 5; i++){
    a++;
}
// (iv) While loop
while (a < 5){
    a++;
}
// (v) Do-While loop
do {
    a++;
} while ( a < 5);
return 0;
}

```

IR Code Snippets in Q1b.1l:

Q1) Write C programs for each of the following constructs, generate the IR, and briefly describe how each is represented. For the loop variants, rewrite the same loop in all three forms and compare the IR.

Preds = %x1 %x2\$ \ldots\$ means that the given block can be reached from blocks x1, x2, \$ \ldots\$

Explanations for each part:

i) if-else:

%5 = load i32, ptr %2, align 4 ; load a

```

%6 = icmp eq i32 %5, 2          ; check: a == 2
br i1 %6, label %7, label %8    ; if (true) -> go to block 7 else -> go to block 8
7:                               ; preds = %0, Label 7 starts here
    store i32 2, ptr %2, align 4 ; do: a = 2 then
    br label %9                 ; go to next part (Block 9)
8:                               ; preds = %0, Label 8 starts here
    store i32 2, ptr %2, align 4 ; do: a = 2 then
    br label %9                 ; go to next part (Block 9)
ii) switch:
9:                               ; preds = %8, %7, Label 9 starts here
    %10 = load i32, ptr %2, align 4 ; load variable a into ptr 10
    switch i32 %10, label %13 [    ; switch(ptr 10 = a): else -> go to 13
        i32 1, label %11          ; if a == 1 -> go to 11
        i32 2, label %12          ; else if a == 2 -> go to 12
    ]
11:                               ; preds = %9, Block 11 starts here
    store i32 2, ptr %2, align 4 ; Set a = 2
    br label %14                 ; Go to block 14
12:                               ; preds = %9, Label 12 starts here
    store i32 2, ptr %2, align 4 ; Set a = 2
    br label %14                 ; Go to block 14
13:                               ; preds = %9, Label 13 starts here
    store i32 2, ptr %2, align 4 ; Set a = 2
    br label %14                 ; Go to block 14

```

iii) for:

```

14:                               ; preds = %13, %12, %11, Label 14 starts here
    %15 = load i32, ptr %2, align 4 ; read value from %2 (variable a), store it in temporary %15
    store i32 %15, ptr %4, align 4 ; store that value into %4

```

```

    br label %16                ; Jump to loop start
16:                             ; preds = %22, %14, Label 16 starts here
    %17 = load i32, ptr %4, align 4 ; Load variable i into %17
    %18 = icmp slt i32 %17, 5      ; Check if i < 5
    br i1 %18, label %19, label %25 ; If i < 5, branch to loop body, else branch to exit loop
19:                             ; preds = %16, Label 19, loop body starts here
    %20 = load i32, ptr %2, align 4 ; Load variable a into %20
    %21 = add nsw i32 %20, 1        ; Store a++ into %21
    store i32 %21, ptr %2, align 4  ; Store the new value of a
    br label %22                  ; Branch to label 22
22:                             ; preds = %19, Label 22 starts here
    %23 = load i32, ptr %4, align 4 ; Load variable i into %23
    %24 = add nsw i32 %23, 1        ; Store i++ into %24
    store i32 %24, ptr %4, align 4  ; Store the new value of i
    br label %16                  ; Branch to label 16
25:                             ; preds = %16, Label 25 starts here
    br label %26                  ; Branch to label 26

```

iv) while:

```

26:                             ; preds = %29, %25, Label 26, while loop starts here
    %27 = load i32, ptr %2, align 4 ; Load the value of variable a into %27
    %28 = icmp slt i32 %27, 5      ; Check if a < 5
    br i1 %28, label %29, label %32 ; Go to loop runner if a < 5, else go to exit loop
29:                             ; preds = %26, Label 29, Loop runner starts here
    %30 = load i32, ptr %2, align 4 ; Load the value of variable a into %30
    %31 = add nsw i32 %30, 1        ; a++
    store i32 %31, ptr %2, align 4  ; Store the new value of a
    br label %26                  ; Branch to label 26 for the next iteration
32:                             ; preds = %26, Label 32, Loop exit starts here
    br label %33                  ; Branch to Label 33

```

v) do-while:

```

33:                             ; preds = %36, %32, Label 33, Do-While loop starts here
    %34 = load i32, ptr %2, align 4 ; Load the value of variable a into %34
    %35 = add nsw i32 %34, 1        ; a++
    store i32 %35, ptr %2, align 4  ; Store the new value of a
    br label %36                  ; Branch to label 36
36:                             ; preds = %33, Label 36 starts here
    %37 = load i32, ptr %2, align 4 ; Load the value of variable a into %37
    %38 = icmp slt i32 %37, 5      ; Check if a < 5
    br i1 %38, label %33, label %39 ; If a < 5, repeat the loop, else exit
39:                             ; preds = %36, Label 39, loop exit starts here
    ret i32 0                     ; int main() returns 0

```

Q2) i) Are for and while identical?

Explanation: Yes, the for and while loops are almost identical in their respective IR's. Both are implemented using a conditional (icmp) and branch (br) block, a loop body, branch instruction to either repeat the loop or exit the loop. The only difference is when we write the for loop in C, we explicitly define the loop iterator variable i, increment steps in the syntax while in the while loop, the compiler takes care of these details.

ii) Where does the condition check appear in the do-while IR compared to the other two?

Explanation: In both the for and while loops the condition check appears before entering the loop body while in the do-while loop they appear after the loop body finishes one iteration.

2.3 C Code in Q1c.c

```
#include <stdio.h>
```

```

int square (int x){
    return x * x;
}
int main(){
    printf("The square of 3 = %d", square(3));
    return 0;
}

```

IR Code Snippets in Q1c.ll:

Q) Locate and explain:

(i) the define and declare keywords

Explanation: The define keyword is present as `define dso_local i32 @square(i32 noundef %0) #0` (The definition of the `int square(int)` function) and as `define dso_local i32 @main() #0` (The definition of the `main()` function)

The declare keyword is present as `declare i32 @printf(ptr noundef, ...) #1` (Tells the compiler the function `printf()` definition is not present in the current file, since it is present in the external library `stdio.h` included as a header in the file)

The define keyword is used to define functions defined in the given file (like `int square (int x)` and `int main()`) itself while the declare keyword is used to declare functions whose definitions are not present in the current file but rather in external files (like `printf()` in the external library `stdio.h` declared at the start of the program) but called from the given file.

(ii) How arguments are passed by value and the return value is conveyed back

Explanation:

```

define dso_local i32 @square(i32 noundef %0) #0 {
    %2 = alloca i32, align 4      ; Allocate memory to store the argument in %0
    store i32 %0, ptr %2, align 4 ; Receives argument x and stores it in %2
    %3 = load i32, ptr %2, align 4
    %4 = load i32, ptr %2, align 4
    %5 = mul nsw i32 %3, %4
    ret i32 %5                  ; Return the value after performing the required calculations
}
%2 = call i32 @square(i32 noundef 3) ; We pass the value 3 into the square() function by value. The
call instruction is used to receive the return value conveyed back by the function.

```

When `main()` calls `square(3)`, the value 3 is passed directly to the `square()` function and received as the parameter `%0`. It is stored in `%2` (allocated using `alloca`), loaded twice and multiplied to get `%5`, which is returned using `ret i32 %5`. In `main()`, the call instruction gives this returned value in register `%2`, which is then passed to `printf` (`%3 = call i32 (ptr, ...) @printf(ptr noundef @.str, i32 noundef %2)`).

(iii) The role of the `ret` instruction.

Explanation:

```

ret i32 %5 ; return result of square
ret i32 0  ; return from main

```

The `ret` function is used to terminate a function, return the control to the caller while also returning value specified. In the `square` function `ret i32 %5` returns the value `x * x` while in the `main` function `ret i32 0` returns the value 0 to the OS.

2.4 C Code in Q1d.c

```
#include <stdio.h>
int main(){
    // (i) Assigning Double to an Int (Narrowing)
    int a = 1;
    double b = 2.0;
    a = (int) b;
    // (ii) Assigning Int to a Long (Widening)
    int c = 3;
    long d = 4;
    d = (long) c;
    return 0;
}
```

IR Code Snippets in Q1d.ll:

Q) Identify the LLVM cast instructions used (fptosi, sext, zext, trunc) and explain when each variant is chosen.

(i) fptosi

Explanation:

```
%6 = load double, ptr %3, align 8
%7 = fptosi double %6 to i32
```

The instruction fptosi (floating point to signed integer) is used to convert a double to an integer (narrowing) where precision is lost in the process of converting a larger type (double) to a smaller type (int).

(ii) sext

Explanation:

```
%8 = load i32, ptr %4, align 4
%9 = sext i32 %8 to i64
```

The instruction sext (sign extension) is used to convert a 32-bit integer to a 64-bit integer (widening) where the sign is preserved while the value is extended.

(iii) zext

Explanation:

This one did not appear in Q1d.c, so a small extra program Q1e.c was written:

```
unsigned char uc = 200;
unsigned int ui = uc;    // widening of an unsigned value
int a = 1, b = 2;
int r = (a < b);         // comparison result (i1) used as an int
%9 = load i8, ptr %2, align 1
%10 = zext i8 %9 to i32   ; unsigned char -> unsigned int
%15 = icmp slt i32 %13, %14 ; the comparison yields an i1
%16 = zext i1 %15 to i32  ; i1 -> int (0 or 1)
```

The instruction zext (zero extension) is used to extend an unsigned value (or an i1 comparison result) to a larger type by filling the higher bits with 0's instead of the sign bit.

(iv) trunc

Explanation:

This one is also from Q1e.c:

```
int i = 300;
```

```
char c = (char) i;      // narrowing
%11 = load i32, ptr %4, align 4
%12 = trunc i32 %11 to i8 ; keep only the low 8 bits (300 -> 44)
```

The instruction `trunc` (truncate) is used to convert a larger integer type to a smaller one by discarding the higher bits, so the value can change (300 becomes 44 in an 8-bit char).

3 Lowering of Data Structures to LLVM IR

3.1 C Code in Q2a.c

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int arr1[5];
    int *arr2 = (int*) malloc(5 * sizeof(int));
    for(int i = 0; i < 5; i++){
        arr1[i] = i;
    }
    int d = arr1[2];
    for(int i = 0; i < 5; i++){
        arr2[i] = i;
    }
    int e = arr2[2];
    return 0;
}
```

IR Code Snippets in Q2a.ll:

Q) In the generated IR, address:

(i) How does the compiler represent array access using `getelementptr` (GEP)? Show one GEP instruction in the report, explaining each operand.

Explanation:

The `getelementptr` (GEP) instruction is used to compute the address of an element in an array. GEP does not perform memory access, it only computes addresses.

Ex: `%16 = getelementptr inbounds [5 x i32], ptr %2, i64 0, i64 %15`

```
[5 x i32]  Array type (5 integers)
ptr %2    Base address of array
i64 0     First index (array itself)
i64 %15   Element index (i)
arr1[i]   -> address = base + i * sizeof(int)  GEP Computes this address
```

(ii) How is the array itself allocated in IR? What does the `alloca` instruction look like for an array type, and how does it differ from a scalar `alloca`?

Explanation:

The `alloca` instruction looks like `%2 = alloca [5 x i32], align 16` for an array type.

The `alloca` instruction allocates memory for the array on the stack. Here, the `[5 x i32]` indicates that the memory allocated is equal to the memory size of 5 integers.

On the other hand, the scalar `alloca` looks like `%1 = alloca i32, align 4` where the memory allocated is equal to the memory size of 1 integer.

(iii) Allocate one array statically and another one dynamically (using `malloc`), and identify the

differences in the generated IR.

Explanation:

Static Allocation: `%2 = alloca [5 x i32], align 16`

Memory equal to the memory size of 5 integers is statically allocated on the stack for the array `arr1`.

```
declare noalias ptr @malloc(i64 noundef) #1
%8 = call noalias ptr @malloc(i64 noundef 20)
store ptr %8, ptr %3
```

Since `malloc()` is an external function, a `declare` instruction is used.

Memory equal to the memory size of 5 integers (5 integers * 4 bytes = 20 bytes) is dynamically allocated on the heap for the array `arr2` using `malloc` which returns a pointer `%8` whose value is then stored in `ptr %3`.

LLVM IR represents array access using the `getelementptr` instruction, which computes addresses based on indices. Static arrays are allocated on the stack using `alloca`, while dynamic arrays are allocated on the heap using `malloc`. This difference is reflected in the IR through distinct allocation mechanisms and pointer types used in element access.

3.2 C Code in Q2bi.c (struct) and Q2bii.cpp (class)

```
#include <stdio.h>
struct Point{
    int x;
    int y;
};
int main(){
    struct Point p;
    p.x = 1;
    p.y = 2;
    return 0;
}
```

ii) C Code in Q2bii.cpp:

```
class Point{
public:
    int x;
    int y;
    // Constructor
    Point(){
        x = 0;
        y = 0;
    }
};
int main(){
    Point p;
    return 0;
}
```

IR Code Snippets in Q2bi.ll and Q2bii.ll:

Q) In the IR for both:

(i) Locate the type definition and state how member alignment is encoded.

Explanation:

Type definition of `struct Point{ int x; int y; }`: `%struct.Point = type { i32, i32 }`

Type definition of class Point{ int x; int y; }:
`%class.Point = type { i32, i32 }`

The type definition shows that the struct/class Point consists of two i32 members (x and y) stored sequentially in memory.

```
%2 = alloca %struct.Point, align 4
store i32 1, ptr %3, align 4
```

Member alignment is not specified in the type definition but is specified during memory allocation (alloca), and memory access (store, load) using (align) keyword. Each i32 is aligned to 4 bytes.

(ii) Are there structural differences in the type layout between struct and class? How does the constructor appear in the class IR?

Explanation:

```
%struct.Point = type { i32, i32 }
%class.Point = type { i32, i32 }
```

There are no structural differences in the type layout between struct and class.

```
call void @_ZN5PointC2Ev(ptr %2)
define void @_ZN5PointC2Ev(ptr %0)
```

Constructors appear as normal functions in LLVM IR. The object pointer (this: %0) is passed as an argument, and the constructor initializes the fields using getelementptr and store instructions. Initializing happens with a constructor call with address of p (Here %2 in the call function).

(iii) How is accessing a member of a class or a struct different from accessing an element from an array?

Explanation:

```
Struct/Class access:
%3 = getelementptr inbounds %struct.Point, ptr %2, i32 0, i32 0
%4 = getelementptr inbounds %struct.Point, ptr %2, i32 0, i32 1
```

Array access: `%15 = getelementptr inbounds [5 x i32], ptr %2, i64 0, i64 %index`

In Struct/Class access, we can access only fixed field indices (compile time constants). The GEP type is %struct.Point while in Array access we can access variable field indices (i, runtime values). The GEP type is [5 x i32].

3.3 C++ Code in Q2c_vector.cpp (std::vector)

```
#include <vector>
int main(){
    std::vector<int> v;
    v.push_back(1);
    v.push_back(2);
    int x = v[1];
    return x;
}
```

IR Code Snippets in Q2c_vector.ll (attributes such as noundef/nonnull/dereferenceable shortened to ...):

```
%"class.std::vector" = type { %"struct.std::_Vector_base" }
%"struct.std::_Vector_base<int, std::allocator<int>>::_Vector_impl_data" = type { ptr, ptr, ptr }
    ; begin, end, end_of_storage
%2 = alloca %"class.std::vector", align 8           ; the vector object itself: three pointers on
```

```

    the stack
call void @_ZNSt6vectorIiSaIiEEC2Ev(ptr ... %2)          ; constructor
invoke void @_ZNSt6vectorIiSaIiEE9push_backEOi(ptr ... %2, ptr ... %3) to label %8 unwind label %14
    ; v.push_back(1)
%10 = call ptr @_ZNSt6vectorIiSaIiEEixEm(ptr ... %2, i64 noundef 1) ; v[1] returns a pointer to
    the element
%11 = load i32, ptr %10, align 4
call void @_ZNSt6vectorIiSaIiEED2Ev(ptr ... %2)          ; destructor
; body of operator[]:
%8 = load ptr, ptr %7, align 8                          ; load the begin pointer
%10 = getelementptr inbounds i32, ptr %8, i64 %9         ; &begin[index]

```

Explanation: `std::vector` is not a built-in type in LLVM IR. It is a normal C++ class, so it is represented as a struct (`%"class.std::vector"`) whose innermost type is three pointers (begin, end and `end_of_storage`). The vector object itself takes only 24 bytes on the stack, while the elements are stored on the heap. Operations like the constructor, `push_back`, `operator[]` and the destructor are all calls to functions, and `operator[]` just loads the begin pointer and uses a `getelementptr` with one index to get the address of the element. The `push_back` calls use `invoke` (instead of `call`) with a landingpad since they can throw an exception.

3.4 C Code in `Q2d_union.c` (union)

```

union Data{
    int i;
    float f;
    char c[8];
};
int main(){
    union Data d;
    d.i = 1;
    d.f = 2.0f;
    return 0;
}

```

IR Code Snippets in `Q2d_union.ll`:

```

%union.Data = type { i32, [4 x i8] }          ; union type: one i32 plus 4 bytes of padding
%2 = alloca %union.Data, align 4              ; size 8, alignment 4
store i32 1, ptr %2, align 4                  ; d.i = 1
store float 2.000000e+00, ptr %2, align 4     ; d.f = 2.0f (same address, different type)

```

Explanation: LLVM IR has no separate union type. A union is represented as a struct with one member (here `i32`) plus padding (`[4 x i8]`). All the members are accessed using the same address `%2`, only the type of the load/store changes (`i32` for `d.i` and `float` for `d.f`).

3.5 C++ Code in `Q2e_struct_class.cpp` (struct vs class)

```

struct S{
    int x;
    int y;
};
class C{
    int x;
    int y;
public:
    void set(){ x = 1; y = 2; }
};
int main(){

```

```

    S s;
    s.x = 1;
    C c;
    c.set();
    return 0;
}

```

IR Code Snippets in Q2e_struct_class.ll:

```

%struct.S = type { i32, i32 }
%class.C = type { i32, i32 }
%2 = alloca %struct.S, align 4
%3 = alloca %class.C, align 4
%4 = getelementptr inbounds %struct.S, ptr %2, i32 0, i32 0 ; s.x
call void @_ZN1C3setEv(ptr ... %3) ; c.set()
; body of C::set():
%4 = getelementptr inbounds %class.C, ptr %3, i32 0, i32 0
store i32 1, ptr %4, align 4
%5 = getelementptr inbounds %class.C, ptr %3, i32 0, i32 1
store i32 2, ptr %5, align 4

```

Explanation: Both S and C have the same layout { i32, i32 }. The only difference is the name of the type (%struct.S and %class.C). The private members of C are not marked anywhere in the IR, and the member function set() is a normal function which takes the pointer to the object as its first argument.

Answers to 3(b), 3(c), 3(d)

3(b) Arrays vs vectors: An array is a built-in type of LLVM IR ([5 x i32]) whose size is fixed at compile time and is allocated directly using alloca. Its elements are accessed using getelementptr with two indices (0, i). A vector (std::vector) is a class, so it is a struct of three pointers (24 bytes) and its elements are stored on the heap. It is accessed only through function calls (push_back, operator[]), where operator[] uses a getelementptr with one index on the begin pointer. Note: LLVM IR also has its own vector type (<4 x i32>) used for SIMD, which is different from std::vector.

3(c) Type of a union: Clang takes the member with the largest alignment (the largest size if there is a tie) as the type of the union and adds padding at the end to match the size of the union. Here i and f need 4 bytes with alignment 4 while c[8] needs 8 bytes with alignment 1, so the type is { i32, [4 x i8] } (size 8, alignment 4).

3(d) struct vs class: There are no differences in the IR once lowered, apart from the name of the type (%struct.S and %class.C). Public/private access is checked by the compiler frontend and is not present in the IR. Both are laid out in the same way and their member functions are normal functions taking the object pointer as an argument. Differences only come from the contents of the type (constructors, virtual functions, base classes) and not from the keyword.

4 Floating-Point Representation in LLVM IR

```

int main(){
    float f = 1.5f;
    double d = 2.5;
    d = d + f;           // fpext + fadd
    f = (float) d * f;    // fptrunc + fmul
    int i = (int) d;      // fptosi
    double e = i;         // sitofp
    if (f < d) e = e / 2.0; // fcmp + fdiv
    e = -e;               // fneg
}

```

```

    return (int) e;
}

```

IR Code Snippets in Q4_float.ll:

```

%2 = alloca float, align 4
%3 = alloca double, align 8
store float 1.500000e+00, ptr %2, align 4
store double 2.500000e+00, ptr %3, align 8
%8 = fpext float %7 to double      ; float -> double
%9 = fadd double %6, %8
%11 = fptrunc double %10 to float   ; double -> float
%13 = fmul float %11, %12
%15 = fptosi double %14 to i32      ; double -> int
%17 = sitofp i32 %16 to double      ; int -> double
%21 = fcmp olt double %19, %20      ; f < d
%24 = fdiv double %23, 2.000000e+00
%27 = fneg double %26

```

Explanation: LLVM IR has float (32 bit), double (64 bit) and others like half, x86_fp80 and fp128. C float and double map to float and double directly. A constant is written in decimal (1.500000e+00) if it can be represented exactly, otherwise as a hexadecimal value (0.1f is written as 0x3FB99999A0000000).

Explanation: The operations supported are fadd, fsub, fmul, fdiv, frem and fneg, and both operands must have the same type. Comparison is done using fcmp which returns an i1 used by br. The predicates are ordered (oeq, olt, ogt, ...) which are false if an operand is NaN, and unordered (une, ult, ...) which are true if an operand is NaN. Clang uses olt for < and une for !=.

Explanation: LLVM IR does not do implicit conversions, so Clang adds an explicit instruction for each one. In $d + f$, the float is first converted to double using fpext, (float) d uses fptrunc, double to int uses fptosi and int to double uses sitofp. When lowered further using llc (Q4_float.s), these become x86-64 SSE instructions: addsd/divsd for double and mulss for float, cvtss2sd/cvtsd2ss for fpext/fptrunc, cvttsd2si/cvtsi2sd for fptosi/sitofp, ucomisd for fcmp, and an xor with the sign bit for fneg.

References

- [1] LLVM Project, "LLVM Language Reference Manual", <https://llvm.org/docs/LangRef.html>
- [2] LLVM Project, "Getting Started with the LLVM System" (directory layout, building from source), <https://llvm.org/docs/GettingStarted.html>
- [3] LLVM Project, "Clang documentation", <https://clang.llvm.org/docs/>
- [4] LLVM Project, "The Often Misunderstood GEP Instruction", <https://llvm.org/docs/GetElementPtr.html>
- [5] Source files and generated outputs of this report: <https://github.com/Rmuk655/Compiler-Construction/tree/main/LLVM-IR-Explorer>